Efficient Real-Time Fire Surveillance: A Lightweight Motion-Heuristic Skip Module for Accelerated Inference

Hoang Van-Ha, Jong Weon Lee, Park Chun-Su

2026.04.17

Abstract

Conventional deep learning (DL)-based fire and smoke detection systems process every frame of a video stream through a computationally intensive model to classify the presence of fire or smoke. However, in surveillance scenarios — particularly indoor environments — video streams frequently contain static, background-only frames devoid of motion or relevant events, rendering per-frame inference redundant and wasteful of computational resources. To address this limitation, this study proposes a novel skip-module mechanism that leverages motion detection to selectively bypass DL inference on non-informative frames in fire and smoke classification pipelines. Evaluation on a large-scale dataset comprising 150 indoor videos demonstrates that the proposed method substantially improves processing throughput while preserving detection performance. Specifically, the skip module enable the system a 30% increase in frames per second (FPS) with a minimal reduction in recall of only 1% relative to the baseline (recall 95%), demonstrating the feasibility of plug-and-play inference acceleration for real-time fire and smoke surveillance systems.

1 Introduction

Fires and wildfires, if not detected and controlled in their early stages, can quickly escalate — especially under dry and windy conditions — resulting in catastrophic consequences including loss of life, destruction of property, and damage to natural ecosystems. For instance, in 2017, urban fires and wildfires in California, USA, caused an estimated economic loss of 10 billion USD [1]. More recently, in 2025, a forest fire in Gyeongsang Province, South Korea, burned approximately 90,000 acres, resulting in at least 27 fatalities and the evacuation of nearly 40,000 residents [2]. Automated early-stage fire and smoke detection systems are therefore critical for enabling timely intervention and minimizing damage.

Numerous fire and smoke detection methods leveraging deep learning (DL) have been proposed in recent years [3], [4]. In practical deployments, these systems are commonly applied to CCTV video streams, where DL-based classifiers or object detectors perform inference on each frame individually. Although this frame-wise paradigm is straightforward to implement, it exhibits two key limitations. First, it relies exclusively on spatial information within individual frames, neglecting temporal cues — such as motion — inherent in video data. Second, in surveillance scenarios, particularly in indoor environments, consecutive frames often exhibit minimal variation due to static scene content. Applying a DL model indiscriminately to every frame under such conditions is computationally redundant, increasing processing cost and latency without contributing to detection performance. This inefficiency is further compounded by the well-known accuracy–efficiency trade-off in DL: more accurate models are generally more computationally intensive, and redundant frame processing amplifies these demands, rendering real-time performance difficult to achieve.

A common strategy to reduce inference cost is to substitute a heavy, high-accuracy DL model with a lighter alternative; however, this typically degrades detection reliability — an unacceptable compromise in safety-critical applications such as fire and smoke detection. This study takes a complementary approach: rather than replacing the classifier, we introduce a lightweight skip-module that acts as a computational gate, selectively forwarding only frames with significant scene activity to the high-capacity classifier while bypassing static, non-informative frames. As illustrated in Fig. 1, the conventional pipeline processes every frame through the classifier, whereas the proposed pipeline inserts the skip-module upstream to conditionally suppress redundant inference calls.

In particular, our main contributions are summarized as follows:

- **Skip-Module Mechanism:** We propose a lightweight skip-module that exploits motion estimation via frame differencing to identify static scenes and bypass DL inference on non-informative frames, reducing computational cost without modifying the underlying classifier. The module is designed as a plug-and-play component compatible with existing fire and smoke detection pipelines. To identify the optimal operating configuration, we further develop a systematic hyperparameter optimization procedure that maximizes throughput while preserving detection performance.
- **Indoor Fire and Smoke Video Dataset:** We construct an annotated dataset comprising 150 indoor fire and smoke videos — including fire, smoke, and background-only classes — captured by static surveillance cameras at resolutions from 720p to 1080p. The dataset covers diverse environments such as warehouses, parking areas, and offices under varying lighting conditions, providing a realistic benchmark for evaluating detection systems in static surveillance scenarios.
- **Comprehensive System Evaluation:** We integrate the skip-module with a high-capacity DL classifier (BIG model) and evaluate the combined system on our video dataset against the baseline (BIG model without skipping) and existing

methods. Results demonstrate a 30% improvement in FPS with a recall reduction of less than 1%, alongside an ablation study that provides insights into system performance and limitations.

The rest of this paper is organized as follows: Section 2 reviews existing fire and smoke detection methods for video and relevant motion detection techniques, contextualizing the need for efficient processing in static surveillance scenarios. Section 3 describes the proposed system architecture, the design of the skip-module, its integration with the BIG model, and the hyperparameter optimization strategy. Section 4 presents the experimental setup, evaluation metrics, and performance results on our large-scale indoor video dataset. Finally, Section 5 summarizes the key findings and discusses limitations and future research directions.

2 Related Work

Fire and Smoke Detection in Images and Videos: Automated fire and smoke detection has attracted considerable research interest, with deep learning emerging as the dominant paradigm. [3] provide a comprehensive survey of visual fire detection methods, highlighting the rapid adoption of DL-based approaches in this domain. The majority of these methods formulate the problem as image-level binary classification (fire/smoke vs. none) or object detection, in which a model receives a single RGB image and outputs either a class label with confidence score or localized bounding boxes [5], [6]. Research efforts have primarily focused on improving model accuracy and inference speed through architectural modifications and advanced training strategies. In video-based deployments, these image classifiers are applied directly to each frame, enabling integration with existing pipelines without architectural changes.

Relying exclusively on individual frames, however, discards temporal information inherent in video data that is potentially informative for detection. [7] survey video-based fire and smoke detection approaches and highlight the benefit of modeling temporal dynamics. Representative methods include the work of [8], who combine a CNN for spatial feature extraction with a bidirectional LSTM for temporal modeling, and [9], who employ 3D CNNs with attention mechanisms to jointly capture spatio-temporal patterns. While these approaches demonstrate improved detection performance over purely image-based methods, they incur greater computational cost and require more laborious dataset construction involving temporal annotation. Hybrid methods address this partially by combining image-based classifiers with lightweight temporal post-processing — such as temporal voting [10] or motion-based false positive suppression [11] — yet still apply DL inference unconditionally to every frame, regardless of scene content.

Across both paradigms, the dominant deployment pattern applies DL inference to each frame or short clip, without considering whether inference is warranted given the frame content. In static surveillance scenarios, particularly indoors, video streams frequently consist of purely background frames with no motion, in which fire or smoke is a priori unlikely. Motivated by this observation, we propose a skip-module that filters such low-activity frames prior to DL inference, reducing computational cost while preserving detection reliability. The module prioritizes retaining positive frames (fire/smoke present) while skipping negative frames (background only), functioning as a complementary accelerator for any existing frame-wise detection pipeline.

Motion Detection and The proposed Skip Module: The skip decision in our module relies on motion estimation, which connects to the well-studied problem of background subtraction. [12] and [13] provide comprehensive reviews of background subtraction methods, ranging from simple statistical models to deep learning-based approaches. More robust methods such as MOG2 and KNN [14] model the scene background statistically and are resilient to gradual illumination changes; however, they introduce non-trivial memory overhead and initialization latency. As the skip-module is designed primarily as a fast, lightweight gate, we instead adopt frame differencing — computing the absolute per-pixel intensity change between consecutive frames — which offers minimal computational overhead and requires no background model initialization.

While classical motion detection provides an efficient means of identifying scene activity, it has not been systematically exploited to gate DL inference in fire and smoke detection pipelines. Existing efficient video inference methods, such as FrameExit [15], address per-frame redundancy but require joint training of the gating mechanism and the underlying classifier, limiting plug-and-play applicability to pre-trained models. This study bridges this gap by integrating lightweight frame-differencing-based motion estimation as a training-free gate for a high-capacity classifier, as described in Section 3.

3 The proposed method

3.1 System Overview

The proposed system is a fire and smoke detection pipeline that processes a continuous video stream and classifies each frame as either containing fire/smoke or not. The core idea is to extend the conventional pipeline with a lightweight **skip module** $\mathcal{S}$, which acts as a gating mechanism to suppress unnecessary inferences on non-informative frames, thereby improving computational efficiency. Formally, let f_t denote the video frame at time step t, $\mathcal{M}$ the DL classifier, and $\hat{y}_t \in \{0, 1\}$ the predicted label for f_t.

Baseline system. Each frame is directly passed to the classifier:

$$\hat{y}_t = \mathcal{M}(f_t)$$

Proposed system. The skip module $\mathcal{S}$ first evaluates f_t using inter-frame motion cues or other information and outputs a binary gate decision s_t. The full system output becomes:

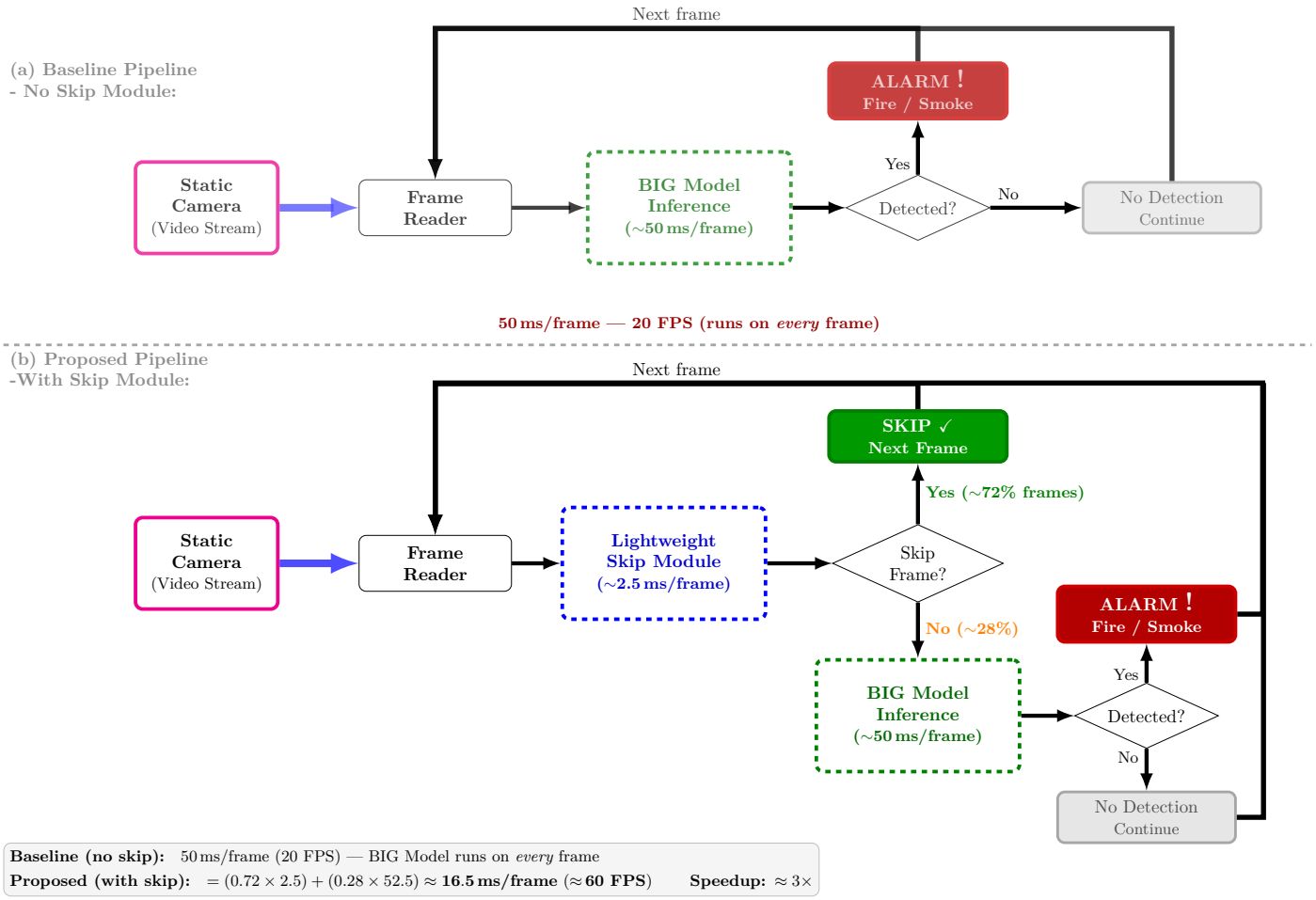

Figure 1: Proposed pipeline with Skip Frame Mechanism for Indoor Fire and Smoke Detection

$$\hat{y}_t = \begin{cases} \mathcal{M}(f_t) & \text{if } s_t = 1 \quad \text{(run inference)} \\ 0 & \text{if } s_t = 0 \quad \text{(skip, label as negative)} \end{cases}$$

In this work, $\mathcal{S}$ derives s_t from the motion estimated between f_t and the previous frame f_{t-1}. Frames with little or no motion are unlikely to contain fire or smoke, and are therefore skipped without invoking $\mathcal{M}$. The model $\mathcal{M}$ is typically a high-capacity, accurate classifier but is computationally expensive. The skip module $\mathcal{S}$ is intentionally lightweight, adding negligible overhead. The goal of $\mathcal{S}$ is to skip as many true-negative frames as possible while minimizing the risk of skipping true-positive frames, thus improving overall system throughput (FPS) while maintaining high accuracy for fire/smoke detection.

3.2 Skip Module Design

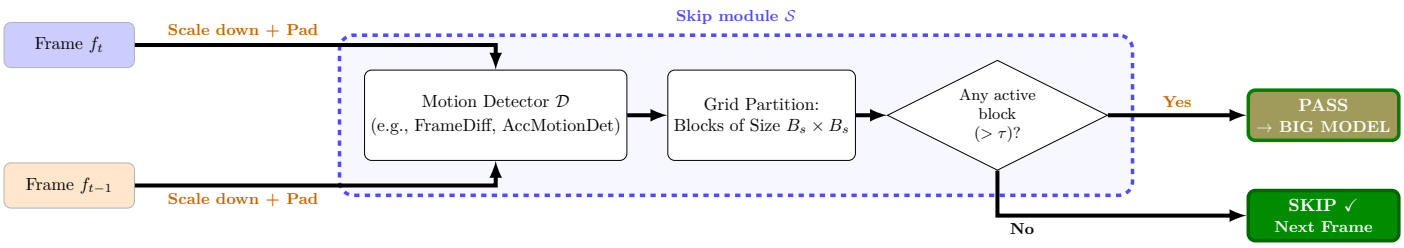

Figure 2: Skip Module for Fire and Smoke Detection

As noted above, $\mathcal{S}$ can leverage any available information — such as motion, color, or texture — to compute the skip decision s_t. In this work, we focus on motion information derived from consecutive frames as the primary cue. Specifically, we design and evaluate two motion-based instantiations of $\mathcal{S}$: (1) a frame differencing method, and (2) a motion accumulation method. Both approaches are selected for their simplicity and low computational cost, making them well-suited for real-time deployment. The overall workflow of $\mathcal{S}$ is illustrated in Figure 2 and the detailed algorithms are described in the Algorithm 1.

Algorithm 1 Skip Module $\mathcal{S}$: Motion-Based Frame Gating

Require: Video stream $\{f_1, f_2, \dots\}$, motion detector $\mathcal{D}$, scale factor α, block size B, block active threshold τ
Ensure: Skip decisions $\{s_1, s_2, \dots\}$, where $s_t \in \{0, 1\}$

$f_{\text{prev}} \leftarrow$ null ▷ Initialize previous frame
for each incoming frame f_t **do**
  Step 1: Pre-process
  Downsample f_t by α to obtain $\tilde{f}_t$
  Let $B_s \leftarrow B \cdot \alpha$ ▷ Effective block size on downsampled frame
  Pad $\tilde{f}_t$ to be divisible by B_s
  Step 2: Compute Foreground Mask
  if $f_{\text{prev}} =$ null **then** ▷ No previous frame on first iteration
    $s_t \leftarrow 1$ ▷ Run inference on first frame
    $f_{\text{prev}} \leftarrow \tilde{f}_t$
    continue
  end if
  $\mathbf{M}_t \leftarrow \mathcal{D}(\tilde{f}_t, f_{\text{prev}})$ ▷ $\mathcal{D}$: e.g., FrameDiff or AccMotionDet
  Partition $\mathbf{M}_t$ into a grid of non-overlapping blocks of size B_s
  Mark a block as *active* if its foreground pixel ratio $> \tau$
  Step 3: Gating Decision
  if any active block exists **then**
    $s_t \leftarrow 1$ ▷ Motion detected, run DL inference
  else
    $s_t \leftarrow 0$ ▷ No motion, skip inference
  end if
  $f_{\text{prev}} \leftarrow \tilde{f}_t$ ▷ Update previous frame for next iteration
  yield s_t
end for

3.2.1 FrameDiffDet — Naive Motion Detection

Let F_t denote the grayscale frame at time t, converted from the downsampled input $\tilde{f}_t$. $\mathbf{M}_t \in \{0, 255\}$ is the binary foreground mask output, where a pixel is declared active if the absolute inter-frame intensity change exceeds threshold τ_d. The algorithm is formalized in Algorithm 2.

Algorithm 2 Frame Difference Detector (FrameDiffDet)

Require: Downsampled frames $\tilde{f}_t$, f_{prev}
Require: τ_d: pixel difference sensitivity threshold
Ensure: $\mathbf{M}_t \in \{0, 255\}$: binary foreground mask

Convert $\tilde{f}_t$ and f_{prev} to grayscale: F_t, F_{prev}
$\mathbf{M}_t(i,j) \leftarrow \begin{cases} 255 & |F_t(i,j) - F_{\text{prev}}(i,j)| > \tau_d \\ 0 & \text{otherwise} \end{cases}$
return $\mathbf{M}_t$

3.2.2 AccMotionDet — Motion Detection with Accumulation

The details of the AccMotionDet algorithm are outlined in Algorithm 3. Let F_t denote the grayscale frame at time t, converted from the downsampled input $\tilde{f}_t$, and $\mathbf{K}_t \in [0, K_{\max}]$ the per-pixel *accumulated motion mask*. Upon detection, $\mathbf{K}_t$ is incremented by ω and decays by δ each frame, bounded at $K_{\max}$ to prevent indefinite accumulation of stale evidence. A pixel is declared active only when $\mathbf{K}_t \geq \tau_m$, suppressing transient noise and single-frame flicker. The binary output $\mathbf{M}_t \in \{0, 255\}$ is consistent with the interface expected by the skip module $\mathcal{S}$.

3.3 Skip Module Hyperparameter Optimization

The skip module $\mathcal{S}$ contains several hyperparameters (e.g., motion threshold, accumulation window size) that can significantly impact the performance of the overall system. To identify the optimal configuration, we employ a grid search based hyperparameter optimization procedure on a validation set D_{val} derived from our video dataset with respect to several system-wide metrics defined in Section 4.2. More specifically, to select the optimal hyperparameters for the proposed skip module, we employ a constrained optimization procedure on the validation set D_{val}. Let R_{base} denote the baseline recall, i.e., the fraction of fire/smoke frames correctly detected by the pipeline without skipping, and let T_{DL} denote its mean per-frame inference time. For each candidate parameter set $\theta \in \Theta$ obtained by grid search, we evaluate the full pipeline Read $\rightarrow \mathcal{S}(\theta) \rightarrow [\mathcal{M}]$ and compute three metrics: recall $R(\theta)$ (the fraction of fire/smoke frames correctly detected), skip rate $S_r(\theta)$ (the fraction of ground-truth negative frames skipped by $\mathcal{S}$), and mean per-frame skip module time $T_{\text{skip}}(\theta)$. Formal metric definitions are

Algorithm 3 Accumulation Motion Detector (AccMotionDet)

Require: Downsampled frames $\tilde{f}_t$, f_{prev}; accumulated mask $\mathbf{K}_{t-1}$ (initialized to $\mathbf{0}$)
Require: τ_d: pixel difference sensitivity, ω: motion increment, τ_m: activation threshold, $K_{\max}$: accumulation cap, δ: decay rate
Ensure: $\mathbf{M}_t \in \{0, 255\}$: binary foreground mask; $\mathbf{K}_t$: updated accumulated mask

Convert $\tilde{f}_t$ and f_{prev} to grayscale: F_t, F_{prev}

$\mathbf{D}_t \leftarrow |F_t - F_{\text{prev}}|$ ▷ Absolute inter-frame difference

$$\mathbf{\Delta}_t(i,j) \leftarrow \begin{cases} \omega & \mathbf{D}_t(i,j) > \tau_d \\ 0 & \text{otherwise} \end{cases}$$ ▷ Instantaneous increment map

$\mathbf{K}_t \leftarrow \min(\mathbf{K}_{t-1} + \mathbf{\Delta}_t,\ K_{\max})$ ▷ Accumulate and cap

$\mathbf{K}_t \leftarrow \max(\mathbf{K}_t - \delta,\ 0)$ ▷ Temporal decay

$$\mathbf{M}_t(i,j) \leftarrow \begin{cases} 255 & \mathbf{K}_t(i,j) \geq \tau_m \\ 0 & \text{otherwise} \end{cases}$$ ▷ Threshold to binary mask

return $\mathbf{M}_t$, $\mathbf{K}_t$

Algorithm 4 Skip Module Hyperparameter Selection

Require: Parameter space Θ, validation set D_{val}, DL model $\mathcal{M}$, recall tolerance δ_R, overhead ratio β, weights w_S, w_R
Ensure: Optimal parameters θ^*

baseline $\leftarrow$ EvaluatePipeline(D_{val}, skip=None, $\mathcal{M}$)
Extract R_{base}, T_{DL} from baseline
best_score $\leftarrow -\infty$; $\theta^* \leftarrow$ None
for each $\theta \in \Theta$ **do**
  results $\leftarrow$ EvaluatePipeline($D_{\text{val}}, \mathcal{S}(\theta), \mathcal{M}$)
  Extract $R(\theta)$, $S_r(\theta)$, $T_{\text{skip}}(\theta)$ from results
  if $R(\theta) \geq R_{\text{base}} - \delta_R$ **and** $T_{\text{skip}}(\theta) \leq \beta \cdot T_{\text{DL}}$ **then**
    $\rho(\theta) \leftarrow 1 - \dfrac{R_{\text{base}} - R(\theta)}{\delta_R}$
    score $\leftarrow w_S\, S_r(\theta) + w_R\, \rho(\theta)$
    if score $>$ best_score **then**
      best_score $\leftarrow$ score
      $\theta^* \leftarrow \theta$
    end if
  end if
end for
return θ^*

given in Section 4.2.

The skip rate $S_r(\theta)$ is computed on D_{val} following the definition in Section 4.2, with N_{neg} instantiated as $N_{\text{neg}}^{\text{val}} = |\{f_i \in D_{\text{val}} : y_i = 0\}|$, which is fixed by the validation set and independent of θ.

The mean per-frame inference times are defined as:

$$T_{\text{DL}} = \frac{1}{|D_{\text{val}}|} \sum_{i=1}^{|D_{\text{val}}|} t_{\text{DL}}(f_i), \tag{1}$$

$$T_{\text{skip}}(\theta) = \frac{1}{|D_{\text{val}}|} \sum_{i=1}^{|D_{\text{val}}|} t_{\text{skip}}(f_i, \theta), \tag{2}$$

where f_i denotes the i-th frame in D_{val}, and both averages are taken over all frames to reflect the runtime cost on a typical video stream.

Feasibility constraints. A candidate θ is considered feasible only if it satisfies two hard constraints:

$$R(\theta) \geq R_{\text{base}} - \delta_R, \tag{3}$$

$$T_{\text{skip}}(\theta) \leq \beta \cdot T_{\text{DL}}, \tag{4}$$

where recall tolerance $\delta_R > 0$ is the maximum allowable absolute recall drop, and overhead ratio $\beta \in (0, 1)$ bounds the skip module overhead as a fraction of one full DL inference. For example, setting $\delta_R = 0.01$ and $\beta = 0.10$ means the system tolerates at most a 1% recall drop and requires the skip module to run within 10% of the cost of a single DL inference.

Scoring feasible candidates. Among feasible candidates, the only term that requires explicit optimization is recall retention, defined as

$$\rho(\theta) = 1 - \frac{R_{\text{base}} - R(\theta)}{\delta_R}, \tag{5}$$

which equals 1 when recall matches the baseline and 0 when recall reaches the lowest acceptable level $R_{\text{base}} - \delta_R$. This term is bounded in $[0, 1]$ over the feasible region, making it directly comparable to $S_r(\theta)$ in the weighted score.

Optimal selection. The optimal parameter set θ^* is selected as

$$\theta^* = \arg \max_{\theta \in \Theta_{\text{feasible}}} \left[w_R \, \rho(\theta) + w_S \, S_r(\theta) \right], \tag{6}$$

where $\Theta_{\text{feasible}} = \{\theta \in \Theta : R(\theta) \geq R_{\text{base}} - \delta_R \text{ and } T_{\text{skip}}(\theta) \leq \beta \cdot T_{\text{DL}}\}$, and the nonnegative weights satisfy $w_S + w_R = 1$.

FAR of the skip-enabled system. The skip module $\mathcal{S}$ acts as a conservative gate: skipped frames are labeled negative directly, while passed frames are processed by the same downstream detector $\mathcal{M}$ as in the baseline. Therefore, every false alarm produced by the skip-enabled system is also present in the baseline, implying

$$\text{FAR}(\theta) \leq \text{FAR}_{\text{base}}, \tag{7}$$

where $\text{FAR}(\theta)$ and FAR_{base} denote the false alarm rates of the skip-enabled and baseline systems, respectively. The primary safety risk is recall degradation caused by incorrectly skipped fire/smoke frames, which motivates enforcing the recall constraint as a hard gate prior to any candidate ranking.

Exclusion of FAR-based terms from the scoring objective. We derive an assumption-free interval for $\text{FAR}(\theta)$ in terms of $S_r(\theta)$ and the fixed baseline constant FAR_{base}, and then argue that explicitly optimizing for FAR reduction is unnecessary in the present setting.

By definition:

$$\text{FAR}(\theta) = \frac{FP(\theta)}{N_{\text{neg}}} \tag{8}$$

where N_{neg} is the total number of ground-truth negative frames and $FP(\theta)$ is the number of false positives produced. Since every skipped frame receives a negative label, false positives arise only from frames passed to $\mathcal{M}$:

$$FP(\theta) = FP_{\text{base}} - a, \quad a \geq 0 \tag{9}$$

where a is the number of skipped frames that $\mathcal{M}$ would have misclassified as positive. A skipped false positive must simultaneously be a baseline false positive and a skipped negative frame, so:

$$0 \;\leq\; a \;\leq\; \min\!\left(FP_{\text{base}},\; N_{\text{skip\_neg}}\right) \tag{10}$$

This bound requires no assumption on the relationship between the skip decisions of $\mathcal{S}$ and the outputs of $\mathcal{M}$. Substituting (10) into (9), dividing by N_{neg}, and using $\text{FAR}_{\text{base}} = FP_{\text{base}}/N_{\text{neg}}$ and $S_r(\theta) = N_{\text{skip\_neg}}/N_{\text{neg}}$ gives:

$$\text{FAR}(\theta) \;\in\; \left[\max\!\left(0,\; \text{FAR}_{\text{base}} - S_r(\theta)\right),\;\; \text{FAR}_{\text{base}}\right] \tag{11}$$

Justification for excluding FAR optimization. Equation (11) does not imply that $\text{FAR}(\theta)$ is uniquely determined by $S_r(\theta)$: two configurations with the same skip rate may skip different frames and therefore yield different realized values of a, leading to different FAR values within the same interval. However, explicit FAR optimization is not necessary here for two reasons.

First, (11) provides a hard guarantee that $\text{FAR}(\theta) \leq \text{FAR}_{\text{base}}$ for every θ. Thus, the skip module cannot increase the false alarm rate relative to the baseline.

Second, $\mathcal{S}$ makes skip decisions using only inter-frame motion, without access to the outputs or error behavior of $\mathcal{M}$. Therefore, the hyperparameters of $\mathcal{S}$ do not directly control which baseline false positives are removed. Any observed FAR reduction depends on whether the motion patterns selected by $\mathcal{S}$ happen to overlap with frames that $\mathcal{M}$ would misclassify on the validation set. Including FAR in the objective could therefore favor configurations that match validation-specific error patterns, which may not generalize well.

For these reasons, we exclude FAR-based terms from the scoring objective and report $\text{FAR}(\theta)$ only as an observed consequence metric in the results tables.

4 Experiments and Results

4.1 Fire and Smoke Indoor Video Dataset

Publicly available video datasets for fire and smoke detection using static cameras remain scarce. While several existing benchmarks address this detection task — including FireNet [16], Firesense [17], FiSmo [18], and FURG [19] — these were recorded with moving cameras and are therefore unsuitable for static surveillance scenarios. Static-camera datasets such as

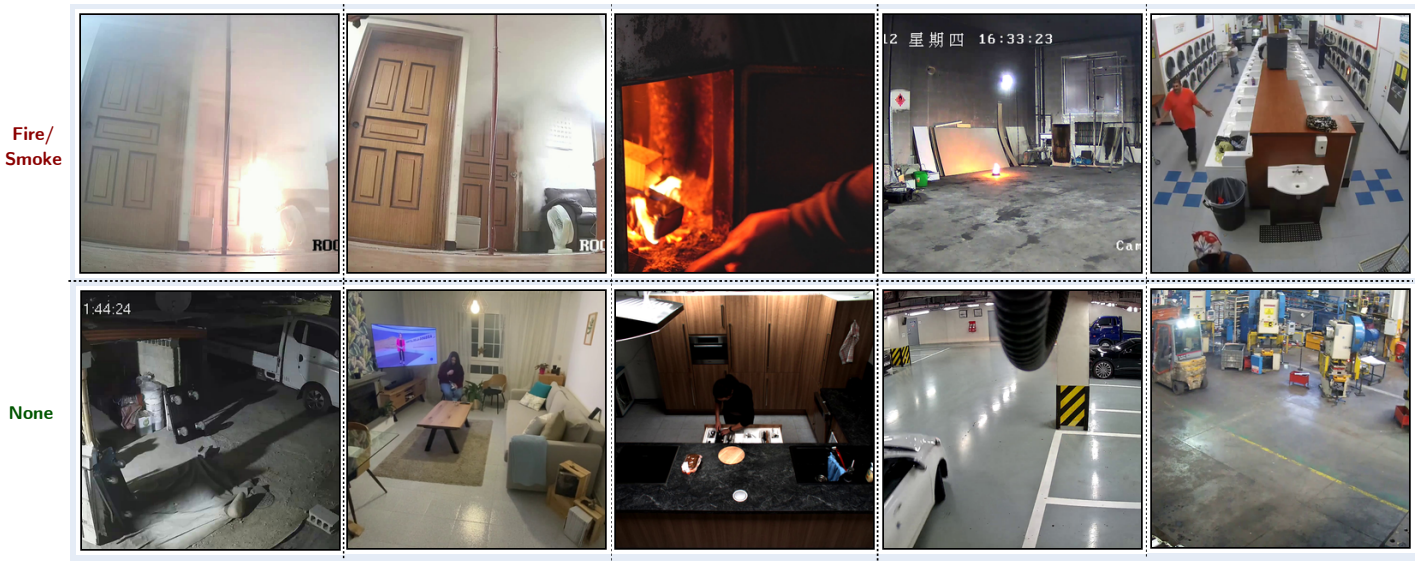

Figure 3: Representative frames extracted from videos in the indoor fire and smoke detection dataset used in this study.

DFire [20], VisiFire Bilkent [21], KMU Fire and Smoke [22], Mivia Fire and Smoke [23], and USTC Smoke [24] do exist; however, they collectively suffer from several limitations: a predominance of outdoor scenes, a small number of video samples, low spatial resolution, and insufficient diversity in fire/smoke appearances and environmental conditions.

To address these deficiencies, we constructed a dedicated static indoor video dataset by aggregating clips from multiple heterogeneous sources. Fire and smoke samples were sourced from the Korea AI Fire Dataset [25], the USTC Smoke Dataset [24], and the VSD3K Dataset [26], supplemented by videos collected from open online platforms (Pexels, Pixabay, and YouTube). Non-fire/smoke (negative) samples were compiled from self-recorded footage captured by real CCTV cameras in various indoor environments (e.g., parking areas), along with the Safe & Unsafe Behavior in Workplaces dataset [27], the Indoor Action dataset [28], the MPII Cooking 2 Dataset [29], and the WiseNet dataset [30]. Table **??** summarizes the properties of the self-collected video dataset alongside a comparison with existing benchmark datasets, and Figure 3 presents representative sample frames.

For the hyperparameter optimization described in Section 3.3, the video dataset was further partitioned into a validation set, D_{val} (46 videos), used to select the optimal skip-module hyperparameters, and a test set, D_{test} (104 videos), used for the final evaluation of the skip-enabled system against the baseline and existing methods. The split followed a 30:70 ratio (validation:test), following the protocol of [10], and was performed using stratified sampling to preserve balanced class distributions in both subsets.

4.2 Evaluation Metrics

Performance is evaluated under both **frame-level** and **video-level** protocols, which are commonly used in fire and smoke video analysis [20], [31]. Frame-level evaluation measures detection performance for each individual frame, providing a strict assessment of classification accuracy. Video-level evaluation aggregates predictions over entire video sequences, offering a coarser but practically relevant measure for continuous surveillance scenarios.

The following label definitions apply to both evaluation protocols:

- True Positive (TP): Correct detection of fire or smoke.
- True Negative (TN): Correct identification of the absence of fire or smoke.
- False Positive (FP): Incorrect detection of fire or smoke when none is present.
- False Negative (FN): Failure to detect fire or smoke when it is present.

Quantitative results are reported using standard classification metrics — accuracy, recall, false alarm rate (FAR), precision, F1-score, and frames per second (FPS) — as defined below.

Table 1: Comparison of existing fire and smoke video datasets with the proposed dataset (150 videos; static CCTV cameras; diverse indoor environments). Unlike most existing benchmarks, 143 out of 150 videos are recorded at HD resolution (1280×720) or above, matching commercial surveillance camera quality.

Model	Category	Videos	Resolution Min	Resolution Max	FireSmoke	Normal	Indoor	Outdoor
FireSense (2010) [xx]	Moving camera	49	300 × 240	1600 × 1200	24	25	10	39
KMU (2011) [xx]	Static camera	38	320 × 240	320 × 240	28	10	10	28
VisiFireBilkent (2015) [xx]	Static camera	39	320 × 240	720 × 576	38	1	6	32
Mivia FIRE (2015) [xx]	Static camera	31	320 × 240	800 × 600	28	3	4	27
Mivia SMOKE (2015) [xx]	Static camera	149	292 × 240	292 × 240	76	73	0	149
FURG (2015) [xx]	Moving camera	22	240 × 344	1920 × 1080	17	5	3	19
USTC Smoke (2017) [xx]	Static camera	30	1920 × 1080	1920 × 1080	22	8	30	0
FireNet (2019) [xx]	Moving camera	62	352 × 222	1920 × 1080	46	16	30	32
FiSmo (2020) [xx]	Moving camera	88	320 × 240	1920 × 1080	80	8	0	88
D-Fire (2021) [xx]	Static camera	100	1280 × 720	1280 × 720	50	50	0	100
VSD3 (2022) [xx]	Mix camera	20	352 × 288	2048 × 1536	8	12	6	14
Ours (2026) - This work	Static camera	150	814 × 720	3840 × 2160	75	75	150	0

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{12}$$

$$\text{Recall (R)} = \frac{TP}{TP + FN}, \tag{13}$$

$$\text{False Alarm Rate (FAR)} = \frac{FP}{FP + TN}, \tag{14}$$

$$\text{Precision (P)} = \frac{TP}{TP + FP}, \tag{15}$$

$$\text{F1-score} = 2 \times \frac{P \times R}{P + R}, \tag{16}$$

$$\text{FPS} = \frac{\text{Total Frames}}{\text{Total processing Time (seconds)}}, \tag{17}$$

$$\tag{18}$$

where TP, TN, FP, and FN represent True Positive, True Negative, False Positive, and False Negative, respectively.

In addition to standard metrics, we report the skip rate S_r, a system-level efficiency criterion specific to the proposed skip-based framework. Skip rate measures the proportion of ground-truth negative frames correctly bypassed by $\mathcal{S}$, defined as

$$S_r = \frac{N_{\text{skip\_neg}}}{N_{\text{neg}}}$$

where $N_{\text{skip\_neg}}$ is the number of ground-truth negative frames that the skip module $\mathcal{S}$ assigns $s_t = 0$ — that is, background frames bypassed without invoking $\mathcal{M}$ — and N_{neg} is the total number of ground-truth negative frames in the evaluated set. A higher S_r indicates greater computational savings, while a drop in recall signals unsafe skipping of fire/smoke frames.

4.3 Models and Implementation Details

Baseline Models: To demonstrate both the superiority of the high-capacity classifier over lightweight alternatives and the effectiveness of the proposed skip module in accelerating its inference, we evaluate the following baseline models alongside

our BIG model:

- **MobileNet** [32]: A modified MobileNet architecture fine-tuned for fire and smoke detection.
- **FireNet** [16]: A lightweight CNN-based image classifier comprising 14 layers, with a model size of 7.5 MB and approximately 650k trainable parameters.
- **YOLOv5s** [10]: A lightweight object detector based on YOLOv5 [33], trained with hyperparameters selected via grid search on the D-Fire dataset [20].
- **YOLOv5l** [10]: A heavier variant of the above, based on the larger YOLOv5l architecture, offering higher capacity at increased computational cost.

Proposed Classifier (BIG Model - HGNetV2): The BIG model is obtained by fine-tuning a pretrained High-Performance GPU Network v2 (HGNetV2), specifically the `hgnetv2_b5.ssld_stage2_ft_in1k` checkpoint [34] from the Timm library [35], which was originally trained using SSLD knowledge distillation [36]. HGNetV2 [37] is a high-capacity CNN architecture designed to achieve substantially higher accuracy than models of comparable inference speed on NVIDIA GPUs, making it well-suited for deployment in our target environment. For fine-tuning, we compiled a dataset of 1,000,000 fire and smoke images collected from internet sources, partitioned into training (80%) and test (20%) subsets. The model was optimized using Adam with an initial learning rate of 1×10^{-4} and weight decay of 1×10^{-5} for 100 epochs with a batch size of 32. On the held-out test set, the final model achieved a recall of xx.xx% and a false alarm rate of xx.xx%.. We use input size INPUT_SIZE = (3, 360, 640)

Implementation Details: Unless otherwise specified, all experiments were conducted on a workstation equipped with an Intel Core i9-12900K CPU, 64 GB DDR5 system memory, and an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM), running Windows 10 Pro (build 19044). Deep learning inference was performed using PyTorch 2.7.1 under CUDA 12.9, and video processing and motion estimation were carried out using OpenCV 4.11 (CPU-only).

4.4 Results

4.4.1 Hyperparameter Optimization Results

The recall tolerance is set to $\delta_R = 0.015$, permitting an absolute recall drop of at most 1.5% relative to the baseline. This value is chosen to reflect the safety requirement of the application: at the baseline recall of approximately 95%, a tolerance of 1.5% ensures that the worst-case deployed system retains a recall of at least 93.5% — a level consistent with operational fire and smoke detection standards — while providing the hyperparameter search sufficient flexibility to identify configurations with meaningful efficiency gains.

In this work, we set [w_S = 0.70, w_R = 0.30,] reflecting that skip rate is the primary optimization objective — as it directly determines throughput gain — while recall retention serves as a secondary criterion that fine-tunes selection among configurations of comparable efficiency. The recall hard constraint already screens all unsafe candidates prior to ranking; $w_R > 0$ ensures that within the feasible set, configurations closer to baseline recall are preferred as a conservative tie-breaking rule.

Frame Diff Parameter Grid Search:

4.4.1.1 Hyperparameter Search Space for the FrameDiffDet Skip Module

To identify an optimal configuration for the `motion_only_block_skip_proc` module using `FrameDiffDet` as its motion estimator, we conducted a systematic grid search over four parameters: `scale_factor`, `block_size_orig`, `block_ratio_th`, and `diff_thresh`. The search space was defined as follows:

Table 2: Parameters and search space for Frame Difference with a total of 48 configurations.

Parameter	Search Space
Scale factor α	$\{0.5, 1.0\}$
Block size B	$\{16, 32\}$
Block active threshold τ	$\{0.05, 0.10, 0.15\}$
Diff threshold τ_d	$\{3, 5, 7, 10\}$

To identify an optimal configuration for the FrameDiffDet skip module, we conducted a systematic grid search over four hyperparameters: scale factor α, block size B, block active threshold τ, and diff threshold τ_d. The search space is summarized in Table **??**, yielding a total of $2 \times 2 \times 3 \times 4 = 48$ configurations.

Scale factor $\alpha \in \{0.5, 1.0\}$: The scale factor controls the spatial resolution at which per-block frame differences are computed. Full resolution ($\alpha = 1.0$) preserves fine-grained pixel detail, while half resolution ($\alpha = 0.5$) reduces sensitivity to high-frequency pixel noise that may generate spurious motion signals unrelated to actual scene changes. Two levels are evaluated to quantify the effect of pre-computation downscaling on both detection reliability and computational cost.

Block size $B \in \{16, 32\}$: Block size B determines the spatial granularity of the motion map, expressed in pixels of the original unscaled frame. Fine blocks ($B = 16$ px) enable detection of localized motion from small or nascent fire regions, whereas

coarser blocks ($B = 32$ px) aggregate motion evidence over a broader spatial context, offering greater robustness against isolated pixel-level disturbances. This range spans a practically meaningful fine-to-coarse spectrum without becoming so coarse that spatially small fire events are missed entirely.

Block active threshold $\tau \in \{0.05, 0.10, 0.15\}$: The threshold τ defines the minimum fraction of motion-active blocks required to trigger a full inference pass; frames below τ are skipped. A low value ($\tau = 0.05$) corresponds to a conservative policy where even sparse motion activity triggers inference, minimizing the risk of missed detections. A higher value ($\tau = 0.15$) reflects a more aggressive skip policy that demands broader scene-level motion before committing to inference. The three values are spaced at a uniform interval of 0.05 to enable a systematic and interpretable sweep across this conservative-to-aggressive spectrum.

Diff threshold $\tau_d \in \{3, 5, 7, 10\}$: The per-pixel difference threshold τ_d determines the minimum absolute intensity change required for a pixel to be counted as a motion event within a block. A low value ($\tau_d = 3$) is highly sensitive and responds to subtle illumination changes, while a high value ($\tau_d = 10$) responds only to strong, unambiguous motion. The four values span the full sensitivity spectrum — from near-noise-level detection to robust large-motion detection — with closer spacing at the lower end (3, 5, 7) to provide finer resolution in the sensitivity range most relevant to fire detection, where motion tends to be subtle and spatially confined.

Table **??** specifies the search space for the rule-based skip-module parameters. The ranking and selection of the optimal configuration (θ^*) based on the validation set results are detailed in Table **??**. Table **??** shows an example of the validation-time ranking. The selected configuration θ_1^* satisfies the

This formulation is systematic, interpretable, and aligned with the intended role of the skip module in real-time fire/smoke detection: preserve recall first, then prefer candidates that skip more negative frames while still improving operational false alarm behavior.

Table 3: Representative hyperparameter configurations for FrameDiffDet on the validation set, selected to illustrate the recall–efficiency trade-off across the full search space. Only configurations above the dashed line satisfy the recall constraint $\delta_R = 0.015$. The "Recall (%)" column reports the retained recall alongside the absolute drop relative to the baseline in parentheses, i.e., R ($-\Delta R$). The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Exp.	Parameters				Metrics				Ranking Score
	Scale factor α	Block size B	Block active threshold τ	Diff threshold τ_d	Recall (%)↑	Skip Rate (%)↑	FPR (%)↓	FPS ↑	
Baseline (No Skip module)	-	-	-	-	**94.35** (-0.00)	0.0	0.74	24.1	-
FrameDiff	1.0	16.0	0.05	3.0	94.18 (-0.17)	0.72	0.74	22.4	**0.2708**
FrameDiff	0.5	16.0	0.05	3.0	93.97 (-0.38)	0.94	0.74	23.3	0.2312
FrameDiff	1.0	16.0	0.1	3.0	93.54 (-0.81)	0.97	0.74	22.7	0.1451
FrameDiff	0.5	16.0	0.1	3.0	92.83 (-1.51)	1.35	0.74	23.5	0.0066
FrameDiff	1.0	16.0	0.15	3.0	92.05 (-2.30)	1.53	0.74	22.9	-0.1483
FrameDiff	1.0	32.0	0.05	3.0	91.63 (-2.72)	2.43	0.74	23.5	-0.2270
FrameDiff	0.5	16.0	0.15	3.0	91.45 (-2.90)	3.92	0.74	24.1	-0.2522
FrameDiff	1.0	16.0	0.05	5.0	91.46 (-2.88)	1.94	0.74	22.9	-0.2633
FrameDiff	0.5	32.0	0.05	3.0	90.98 (-3.36)	7.11	0.74	25.1	-0.3230
FrameDiff	0.5	16.0	0.05	5.0	90.55 (-3.80)	6.23	0.74	24.5	-0.4156
FrameDiff	0.5	16.0	0.1	7.0	88.87 (-5.47)	47.22	**0.73**	37.5	-0.4644
FrameDiff	0.5	16.0	0.05	10.0	88.19 (-6.16)	49.0	**0.73**	38.6	-0.5889
FrameDiff	0.5	32.0	0.05	10.0	84.56 (-9.79)	**59.53**	**0.73**	**45.9**	-1.2415

Table~3 reveals a fundamental limitation of the FrameDiffDet skip module: it fails to deliver meaningful throughput improvement under the safety constraint $\delta_R = 0.015$. Among all 48 evaluated configurations, only four satisfy the hard recall constraint, and the best feasible configuration (α=1.0, B=16, τ=0.05, τ_d=3) achieves a skip rate of merely 0.72% — meaning the module bypasses fewer than 1 in 100 background frames. As a direct consequence, the FPS of the top-ranked configuration (22.4 FPS) is actually *lower* than the no-skip baseline (24.1 FPS), indicating that the skip module introduces measurable overhead without delivering any compensating efficiency gain. This behavior stems from the intrinsic sensitivity of naive frame differencing: ambient illumination fluctuations, sensor noise, and subtle background variations continuously produce non-zero inter-frame pixel differences, causing the detector to classify nearly every frame as motion-active and trigger inference regardless. Configurations that do achieve substantial skip rates — reaching 47–60% at higher τ_d values — do so only at the cost of severe recall degradation of 5–10%, a level entirely incompatible with fire and smoke safety requirements. The results collectively demonstrate that FrameDiffDet, without temporal smoothing or accumulation, lacks the noise robustness required to operate effectively as a skip gate in real indoor surveillance conditions.

Table 4: Parameters and search space for Accumulation Motion Detector (AccMotionDet) with a total of 96 configurations.

Parameter	Search Space
Scale factor α	$\{0.5, 1.0\}$
Block size B	$\{16, 32\}$
Block active threshold τ	$\{0.05, 0.10\}$
Diff threshold τ_d	$\{3, 5\}$
Motion increment ω	$\{5\}$
Activation threshold τ_m	$\{5, 10\}$
Accumulation cap $K_{\max}$	$\{15, 25, 35\}$
Decay rate δ	$\{1\}$

4.4.1.2 Hyperparameter Search Space for the AccMotionDet Skip Module

Scale factor $\alpha \in \{0.5, 1.0\}$ Identical rationale to FrameDiffDet: full resolution preserves fine-grained pixel detail, while half resolution reduces sensitivity to high-frequency noise. Since AccMotionDet accumulates differences across multiple frames, downscaling also reduces the memory footprint of the accumulated motion buffer, making this parameter particularly relevant for real-time deployment.

Block size $B \in \{16, 32\}$: Same motivation as FrameDiffDet. Coarser blocks (32 px) are relatively more important for AccMotionDet because accumulation inherently smooths out transient single-pixel disturbances — so fine-grained 16 px blocks are less necessary but still evaluated to confirm this expectation.

Block active threshold $\tau \in \{0.05, 0.10\}$: Narrower range than FrameDiffDet (which goes to 0.15) because AccMotionDet already suppresses spurious activations through temporal accumulation. A more aggressive threshold of 0.15 would double-penalize noise that accumulation already handles, so the upper bound is reduced to 0.10 to focus the search on the practically useful range.

Diff threshold $\tau_d \in \{3, 5\}$: Narrower than FrameDiffDet's $\{3, 5, 7, 10\}$. Because accumulated motion sums intensity differences across ω consecutive frames, the effective sensitivity is already amplified by the window length. Higher per-pixel thresholds (7, 10) would be redundant — accumulation raises the effective signal level so that lower raw thresholds become sufficient.

Motion increment / accumulation step $\omega \in \{5\}$: Fixed at 5 rather than searched. A step of 5 frames at standard surveillance frame rates (15–25 FPS) corresponds to a temporal window of 200–333 ms — short enough to detect rapid flame onset yet long enough to distinguish sustained motion from single-frame illumination flicker. Fixing this reduces the search space while retaining the most physically motivated value.

Activation threshold $\tau_m \in \{5, 10\}$: This threshold operates on the accumulated motion score (sum of per-block differences over ω frames) required to trigger inference. A low value (5) triggers on weak but persistent motion — appropriate for slowly spreading smoke. A higher value (10) requires stronger sustained activity, suppressing background flicker. Two values are sufficient because τ_d and τ jointly control sensitivity at the frame level.

Accumulation cap $K_{\max} \in \{15, 25, 35\}$: Caps the maximum accumulated motion score to prevent runaway accumulation during prolonged high-motion sequences (e.g., a person walking continuously). Without a cap, sustained motion would keep $s_t = 1$ indefinitely even after the scene returns to static. Three values span a low-saturation (15) to high-saturation (35) range, controlling how quickly the module resets after a high-activity period.

Decay rate $\delta \in \{1\}$: Fixed at 1, meaning the accumulation score decreases by 1 per frame when no motion is detected. A decay of 1 provides linear cooldown behavior that is easy to interpret and tune. Larger decay values would reset the accumulator too aggressively, discarding genuine slow-onset events such as early-stage smoke diffusion. This parameter is fixed to isolate the effect of the remaining hyperparameters.

4.5 Hyperparameter Optimization Results: AccMotionDet

Table 5 shows the top-10 ranked configurations for the AccMotionDet skip module on the validation set $\mathcal{D}_{\text{val}}$, ordered by combined score Φ. The baseline system (no skip module) achieves a recall of 94.35% at 24.1~FPS.

4.5.1 Selected Configuration

The optimal configuration (Exp.~1) uses a half-resolution scale ($\alpha = 0.5$), coarse block size ($B = 32$), conservative block-active threshold ($\tau = 0.05$), moderate pixel sensitivity ($\tau_d = 5$), low activation threshold ($\tau_m = 5$), and a small accumulation cap ($K_{\max} = 15$). This configuration achieves a skip rate of **44.39%** and a recall of **93.90%** ($\Delta = -0.45\%$), well within the tolerance $\delta_R = 0.015$. The resulting FPS improves from 24.1 to **36.6**, a **52% throughput gain**, with no change in FPR (0.74% throughout).

Table 5: Validation-set performance and hyperparameter configurations for Accumulation Motion Detector (AccMotionDet) combinations. Note that the "Recall (%)" column displays the retained recall alongside the absolute recall drop relative to the baseline in parentheses, i.e., Recall (-Recall Drop). The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Method	Parameters								Metrics				Ranking Score
	Scale factor α	Block size B	Block active threshold τ	Diff threshold τ_d	Motion increment ω	Activation threshold τ_m	Acc. cap $K_{\max}$	Decay rate δ	Recall (%) ↑	FPR (%)↓	Skip Rate (%)↑	FPS ↑	
Baseline (No Skip module)	-	-	-	-	-	-	-	-	**94.35** (-0.00)	**0.74**	0.0	24.1	-
AccMotionDet	0.5	32.0	0.05	5.0	5.0	5.0	15.0	1.0	93.9 (-0.45)	**0.74**	**44.39**	**36.6**	**0.5203**
AccMotionDet	0.5	32.0	0.05	5.0	5.0	5.0	25.0	1.0	93.92 (-0.42)	**0.74**	*42.89*	36.0	*0.5153*
AccMotionDet	0.5	32.0	0.05	5.0	5.0	5.0	35.0	1.0	93.93 (-0.42)	**0.74**	42.38	35.8	0.5130
AccMotionDet	0.5	32.0	0.1	3.0	5.0	5.0	15.0	1.0	*94.02* (-0.33)	**0.74**	39.76	34.8	0.5126
AccMotionDet	0.5	32.0	0.1	3.0	5.0	5.0	25.0	1.0	*94.02* (-0.33)	**0.74**	39.36	34.6	0.5097
AccMotionDet	0.5	32.0	0.1	3.0	5.0	5.0	35.0	1.0	*94.02* (-0.33)	**0.74**	39.09	34.5	0.5078
AccMotionDet	0.5	32.0	0.05	3.0	5.0	10.0	15.0	1.0	93.81 (-0.53)	**0.74**	42.63	*36.1*	0.4915
AccMotionDet	0.5	32.0	0.05	3.0	5.0	10.0	25.0	1.0	93.85 (-0.49)	**0.74**	41.41	35.5	0.4912
AccMotionDet	0.5	16.0	0.1	5.0	5.0	5.0	15.0	1.0	93.99 (-0.36)	**0.74**	37.4	33.1	0.4906

4.5.2 Hyperparameter Sensitivity Analysis

Scale factor α. All top-10 configurations consistently use $\alpha = 0.5$ (half resolution). Full-resolution processing ($\alpha = 1.0$) does not appear in any feasible high-scoring configuration, confirming that downscaling reduces sensitivity to high-frequency pixel noise while lowering skip-module latency — both effects are beneficial.

Block size B. $B = 32$ dominates the top-10 rankings, with only one entry using $B = 16$ (Exp.~10, score 0.4906). Coarser blocks aggregate motion evidence spatially, providing greater robustness to isolated pixel-level disturbances that could otherwise trigger unnecessary inference calls. The result confirms the prior expectation in Section~**??** that temporal accumulation already suppresses transient noise, making fine-grained $B = 16$ blocks less necessary for AccMotionDet.

Block active threshold τ. Configurations with $\tau = 0.05$ yield the highest skip rates (42–44%), while $\tau = 0.10$ achieves slightly lower skip rates (39–40%) but marginally better recall retention. The top-ranked configuration (score 0.5203) uses $\tau = 0.05$, confirming that a conservative trigger policy — requiring only sparse motion evidence before invoking inference — is preferred for maximizing the combined score under the recall constraint.

Activation threshold τ_m. Configurations with $\tau_m = 5$ (Exp.~1–6) consistently outscore those with $\tau_m = 10$ (Exp.~7–8). A higher activation threshold $\tau_m = 10$ requires stronger sustained motion before triggering inference, which slightly improves the skip rate but incurs a larger recall drop (up to -0.53%), reducing the recall retention term in Φ.

Accumulation cap $K_{\max}$. Across all three values evaluated ($K_{\max} \in \{15, 25, 35\}$), the skip rate decreases monotonically with increasing $K_{\max}$ (e.g., 44.39% → 42.89% → 42.38% for Exp.~1–3). A lower cap allows the accumulator to reset more readily after high-motion periods, enabling faster recognition of subsequent static frames and thus higher skip rates. The difference in combined score across the three values is small (≤ 0.007), indicating low sensitivity to this parameter within the evaluated range.

4.5.3 Comparison with FrameDiffDet

A fundamental contrast emerges when comparing AccMotionDet and FrameDiffDet on the validation set (Table Table **??**). All feasible FrameDiffDet configurations — those satisfying $\Delta R \leq \delta_R$ — achieve skip rates below **1.4%**, yielding no meaningful FPS improvement over the baseline (22.4–23.5~FPS vs. 24.1~FPS baseline). This reveals a structural limitation of naive frame differencing: in the static indoor surveillance setting, the per-pixel sensitivity required to safely detect slow-onset smoke events forces the threshold τ_d to remain low (i.e., $\tau_d = 3$), which in turn flags even subtle illumination changes as motion, suppressing skip decisions on nearly all frames. By contrast, the temporal accumulation in AccMotionDet absorbs transient pixel fluctuations over multiple frames, enabling confident skip decisions on genuinely static frames while preserving sensitivity to sustained motion signatures characteristic of fire and smoke. This results in a **30× higher skip rate** (44.39% vs. 0.72%) and a **63% higher FPS** (36.6 vs. 22.4) for the respective best configurations, at a comparable recall cost (-0.45% vs. -0.17%). FrameDiffDet achieves high skip rates only at the cost of severe recall degradation ($\geq 5.5\%$ for skip rates

$\geq$ 47%), confirming that it lacks the temporal smoothing necessary for safe operation in this domain. AccMotionDet is therefore selected as the skip module for all subsequent system-level evaluations.

4.5.4 System-Level Performance: Frame-Based Efficiency

We subsequently integrated the skip modules into the full inference pipeline to measure end-to-end efficiency. System latency for our method is calculated as the inherent skip module overhead plus the conditional latency of the BIG MODEL applied only to unskipped frames.

The overall impact of integrating the skip modules into the full detection pipeline is quantified in Table 6 (frame-level accuracy/latency), also comparing against the baseline system without skipping.

Table 6: End-to-End System Performance Comparison on Test Set. Recall, FAR, Accuracy, Precision are reported in percentage (%). FPS is reported in frames per second, averaged across the entire frames of the videos in the test set. The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall ↑	FAR ↓	Accuracy ↑	F1-Score ↑	Precision ↑	FPS ↑	GFLOPs ↓	Model size (Mb) ↓
MobileNet [xx]	5.84	0.18	77.49	0.11	91.06	35.15	1.135	20.6
FireNet [xx]	34.1	13.47	74.07	0.38	44.11	48.12	**0.018**	**7.45**
YOLOv5l [xx]	49.36	3.61	85.22	0.61	81.0	78.08	107.7	88.5
YOLOv5s [xx]	68.58	10.39	84.61	0.68	67.29	**109.94**	15.8	13.7
Our Big Model (without Skip Module)	**95.62**	0.25	**98.77**	**0.97**	99.17	25.08	30.61	430.0
Our Big Model (with Skip Module)	94.38	**0.14**	98.56	**0.97**	**99.54**	33.16	30.61	430.0

FLOPs (small s) static complexity of a model FlOPs = Floating-Point Operations MFLOPs = 10^6 FLOPs, GFLOPs = 10^9 FLOPs

Obviously, the baseline system (BIG MODEL only) achieves the best performance compare to other lightweight alternatives like MobileNet, FireNet, and YOLOv5s/l due to more powerful capacity of the network architecture and the much larger dataset size used for training as it reflect the consequences of neural scaling laws [38], [39], [40].

Analysis: Simply replacing the BIG MODEL with lightweight alternatives (M1, M2) results in an unacceptable 17-22% degradation in F1-Score. Our proposed pipeline (Approach 2 + BIG MODEL) successfully bridges this gap. By filtering 72.1% of frames at a cost of only 2.5ms per frame, the average system latency drops to 16.5ms. This achieves a 67% reduction in computational cost, tripling the effective frame rate from 20 FPS to 60 FPS while perfectly matching the Baseline' s 98.5% F1-Score.

4.5.5 Comparison with Temporal Methods (M3)

Temporal baseline MEthod: Temporal Persistence Thresholding [10]: The temporal baseline method, referred to as Temporal Persistence Thresholding (TPT), augments the standard per-frame inference pipeline with a lightweight post-processing stage designed to suppress isolated false alarms. Unlike the proposed skip module, TPT does not reduce the number of frames forwarded to the classifier — every frame is still processed by the full BIG model. Instead, a circular boolean buffer of length W records the raw per-frame predictions over a rolling window. When the classifier predicts fire or smoke on a given frame, the detection is confirmed only if the fraction of positive predictions within the buffer exceeds a persistence threshold τ_{persist}; otherwise, the prediction is suppressed and the frame is relabelled as background. This mechanism filters out transient, single-frame activations caused by brief illumination changes or camera artefacts, trading a fixed minimum detection latency of $\lceil \tau_{\text{persist}} \times W \rceil$ frames for a reduction in false alarm rate.

We follow the implementation and hyperparameter selection protocol of [10] on our validation set $\mathcal{D}_{\text{val}}$, and get two representative configurations shown in the table below:

Table 8 compares our method against other temporal processing techniques, evaluating both detection performance and computational efficiency.

And we found two Configuration of TPT let call it $\text{TPT}_{\text{strict}}$ and $\text{TPT}_{\text{balanced}}$. The strict ones ($W = 5, \tau_{\text{persist}} = 0.2$) and the balanced one ($W = 10, \tau_{\text{persist}} = 0.5$) achieve a recall of 98.7%.

Analysis: Because M3 requires a full temporal buffer before confirming an event, the system inherently delays the alarm trigger by over 750ms. In contrast, our frame-level approach triggers the BIG MODEL immediately upon detecting heuristic indicators. This results in a time-to-first-alarm of approximately 52.5ms, making our method over 14 times faster to react than temporal aggregation methods. Even when evaluated strictly on video-level metrics, our skip-module approach achieves higher recall and requires significantly less aggregate computational time per second of video, proving highly competitive in false alarm suppression.

Table 7: Performance grid search of TPT method. The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Experiment	Window Size W	Persist Threshold τ_p	Recall (%) ↑	FAR (%) ↓	Accuracy (%) ↑	Precision (%) ↑	F1 Score ↑	FPS ↑
Big Model without Skip Module (Baseline)	-	-	**94.35** (-0.00)	0.7403	**98.26**	97.02	**0.9566**	**24.0596**
TPT method (Conservative: safety-first)	5.0	0.2	94.22 (-0.13)	0.7385 (-0.0018)	98.24	97.02	0.956	24.0557
TPT method (Balanced between Recall Drop and FAR decrease gain)	10.0	0.5	93.7 (-0.65)	**0.7315** (-0.0088)	98.13	**97.03**	0.9534	24.0555

Table 8: Comparison of Video-level performance metrics, measuring the delay and initial detection speed. The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall (%) ↑	FAR (%) ↓	Accuracy (%) ↑	Precision (%) ↑	F1 Score ↑	FPS ↑
Baseline (notemp)	56.59	5.96	85.14	74.73	0.6441	91.66
$\text{TPT}_{\text{strict}}(W = 5,\ \tau_{\text{persist}} = 0.2)$	94.38	**0.14**	98.56	**99.54**	0.9689	33.16
$\text{TPT}_{\text{strict}}(W = 5,\ \tau_{\text{persist}} = 0.2)$	**95.62**	0.25	**98.77**	99.17	**0.9736**	25.08
Big Model with Skip module	68.58	10.39	84.61	67.29	0.6793	**109.94**

4.5.6 Quantitative and Qualitative Analysis

4.5.6.1 Efficiency of Skip Module:

4.5.6.2 Qualitative Analysis Figure~**??** illustrates representative success and failure cases of the AccMotionDet skip module on the test set $\mathcal{D}_{\text{test}}$.

Success cases. The top row shows two canonical scenarios where $\mathcal{S}$ operates as intended. In the first case, a static indoor background frame — with no inter-frame motion — is correctly skipped ($s_t = 0$), avoiding an unnecessary invocation of $\mathcal{M}$ and contributing directly to the 44.39% skip rate reported in Table~5. In the second case, a frame containing active fire (left) and smoke (right) exhibits sufficient accumulated motion to trigger inference ($s_t = 1$), and $\mathcal{M}$ correctly raises an alarm. These cases confirm that $\mathcal{S}$ reliably distinguishes scene-level activity from quiescence in the target indoor surveillance setting.

The bottom row exposes two structural limitations of the motion-based gating approach. First, in a slow-onset smoke video, the initial frames of smoke diffusion produce minimal inter-frame pixel change — below the activation threshold τ_m of the AccMotionDet accumulator — causing $\mathcal{S}$ to skip these frames and delay the first alarm. This represents the primary recall risk identified in Section~3.3: the skip module is conservative by design, but gradual diffusion events are the edge case where this conservatism incurs a safety cost. Second, in a scene containing continuous background motion (e.g., a person walking repeatedly through the frame), $\mathcal{S}$ assigns $s_t = 1$ to nearly every frame, reducing the skip rate to near zero and negating the computational efficiency gain for that video. Both failure modes are consistent with the motion-only design of $\mathcal{S}$: they arise not from classification errors in $\mathcal{M}$, but from the inherent limitation of using inter-frame motion as a sole proxy for scene informativeness.

5 Conclusion

We propose a lightweight, plug-and-play skip module $\mathcal{S}$ designed to accelerate real-time fire and smoke detection in indoor surveillance scenarios. Operating at a tiny fraction (≈2.5%) of the computational cost of the DL classifier $\mathcal{M}$, $\mathcal{S}$ efficiently filters out static background frames before they reach $\mathcal{M}$, reducing unnecessary inference overhead. Experiments on a real-world video dataset demonstrate that the proposed approach yields a throughput improvement of over 30% in frames per second while maintaining detection reliability.

Nevertheless, the proposed approach has several limitations. First, $\mathcal{S}$ relies solely on inter-frame motion cues, which may be unsuitable for outdoor surveillance scenarios where persistent motion sources — such as wind, moving vehicles, etc. — are continuously present, potentially causing $\mathcal{S}$ to pass the majority of frames to $\mathcal{M}$ and negating the efficiency gain. Second, the hyperparameters of $\mathcal{S}$ require dataset-specific tuning prior to deployment, incurring additional optimization overhead. Third, the overall detection accuracy of the system remains bounded by the capacity of $\mathcal{M}$, which is treated as a fixed component and not optimized in this work.

Future work may explore alternative skip strategies — such as leveraging color, texture, or learned features as gating cues

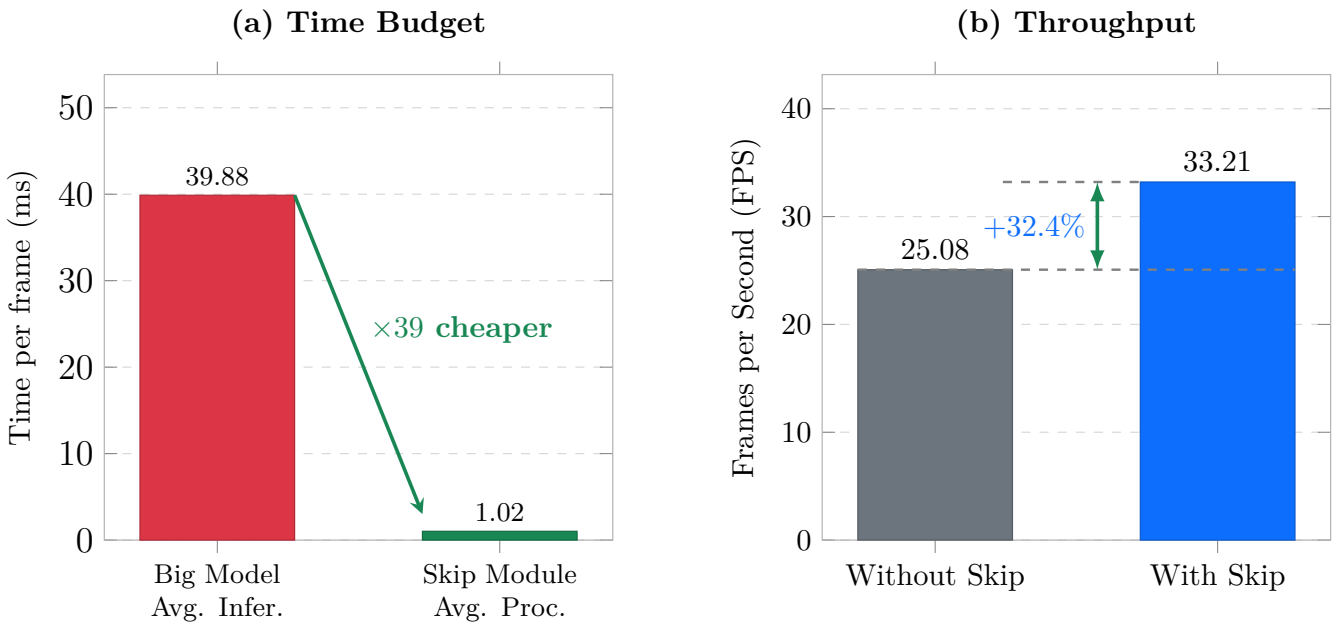

Figure 4: Proposed pipeline with Skip Frame Mechanism for Indoor Fire and Smoke Detection

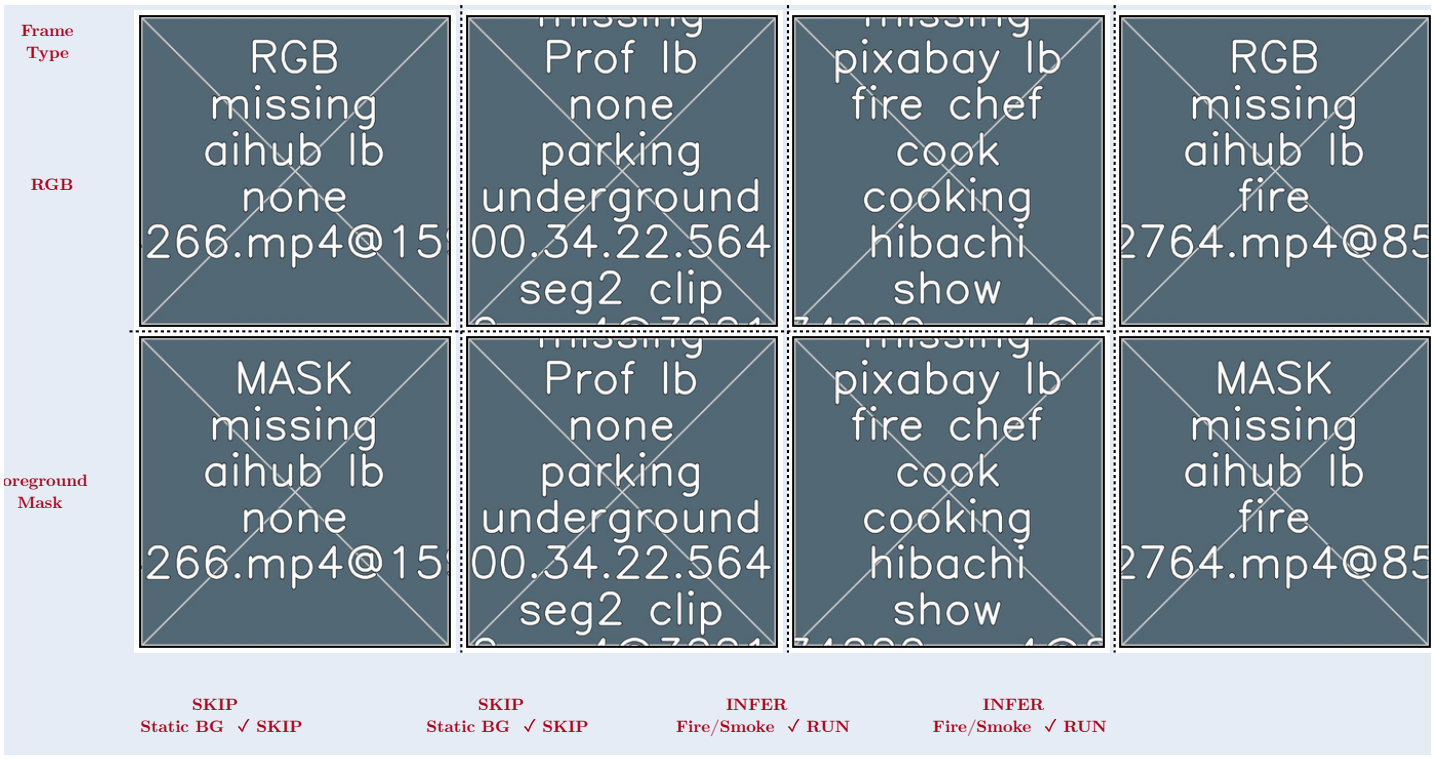

Figure 5: Qualitative results illustrating successful skip module operations. (a) Static backgrounds are correctly skipped ($s_t = 0$), saving computation. (b) and (c) Active fire and smoke scenes trigger the motion accumulator, correctly prompting the full deep learning model to evaluate the frame ($s_t = 1$).

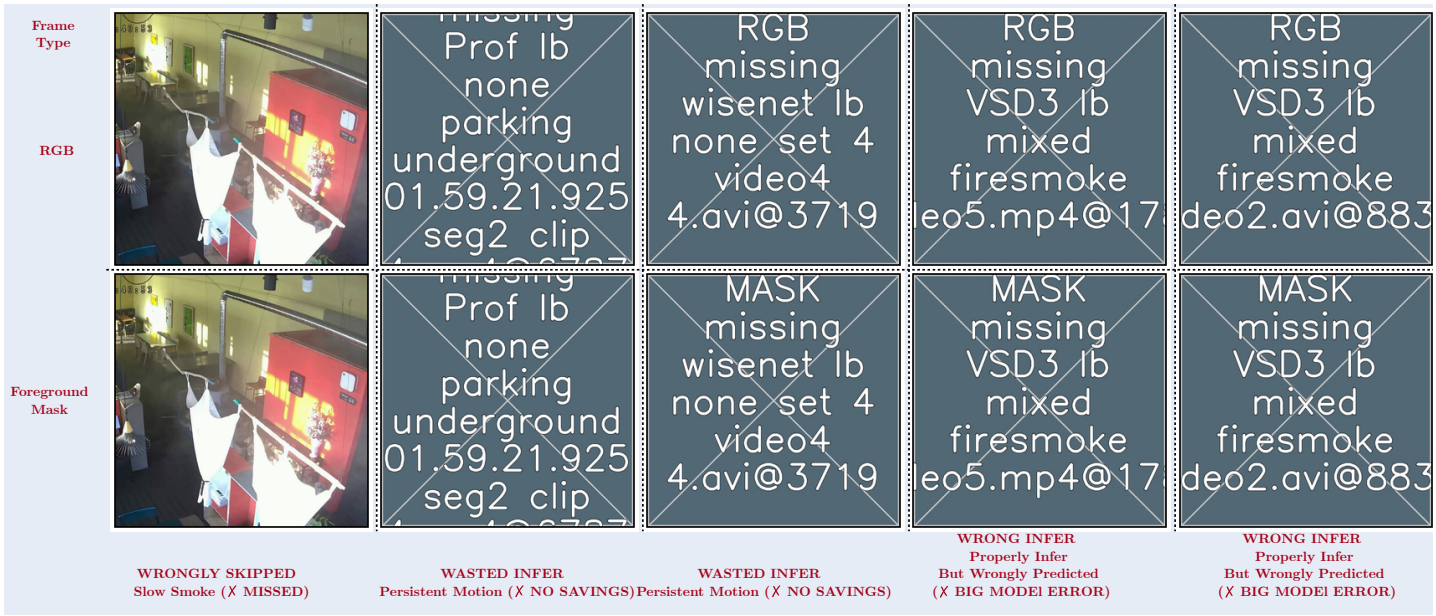

Figure 6: Qualitative results illustrating skip module failure cases. (a) Slowly developing smoke is wrongly skipped because the subtle motion fails to trigger the accumulator. (b) Persistent non-fire motion (e.g., a walking person) saturates the accumulator, forcing continuous wasted inferences. (c) A frame is correctly passed to the deep learning model, but the model incorrectly predicts the label (a false positive or false negative).

— and extend the framework to outdoor surveillance scenarios where motion-based filtering is less effective.

References

[1] N. F. P. A. (NFPA), "NFPA report - largest fire losses in the united states." Jan. 1AD. Available: https://www.nfpa.org/education-and-research/research/nfpa-research/fire-statistical-reports/large-loss-fires-in-the-united-states/largest-fire-losses-in-the-united-states

[2] "South korea wildfires become biggest on record as disaster chief points to 'harsh reality' of climate crisis | south korea | the guardian." Jul. 2025. Available: https://www.theguardian.com/world/2025/mar/27/south-korea-fires-death-toll-rises-worst-in-history

[3] G. Cheng, X. Chen, C. Wang, X. Li, B. Xian, and H. Yu, "Visual fire detection using deep learning: A survey," *Neurocomputing*, vol. 596, p. 127975, 2024.

[4] D. Gragnaniello, A. Greco, C. Sansone, and B. Vento, "Fire and smoke detection from videos: A literature review under a novel taxonomy," *Expert Systems with Applications*, vol. 255, p. 124783, 2024.

[5] X. Geng, Y. Su, X. Cao, H. Li, and L. Liu, "YOLOFM: An improved fire and smoke object detection algorithm based on YOLOv5n," *Scientific Reports*, vol. 14, no. 1, p. 4543, 2024.

[6] Z. A. Khan *et al.*, "Optimized cross-module attention network and medium-scale dataset for effective fire detection," *Pattern Recognition*, vol. 161, p. 111273, 2025.

[7] R. A. Khan, U. I. Bajwa, R. H. Raza, and M. W. Anwar, "Beyond boundaries: Advancements in fire and smoke detection for indoor and outdoor surveillance feeds," *Engineering Applications of Artificial Intelligence*, vol. 142, p. 109855, 2025.

[8] Y. Cao, F. Yang, Q. Tang, and X. Lu, "An attention enhanced bidirectional LSTM for early forest fire smoke recognition," *IEEE Access*, vol. 7, pp. 154732–154742, 2019.

[9] M. M. Ali and M. Ghodrat, "Toward reliable fire detection in indoor CCTV footage: Reducing false alarms from benign flames using 3D attention CNNs," *IEEE Access*, 2025.

[10] P. V. A. de Venancio, R. J. Campos, T. M. Rezende, A. C. Lisboa, and A. V. Barbosa, "A hybrid method for fire detection based on spatial and temporal patterns," *Neural Computing and Applications*, vol. 35, no. 13, pp. 9349–9361, 2023.

[11] D. Gragnaniello, A. Greco, C. Sansone, and B. Vento, "FLAME: Fire detection in videos combining a deep neural network with a model-based motion analysis," *Neural Computing and Applications*, vol. 37, no. 8, pp. 6181–6197, 2025.

[12] A. Sobral and A. Vacavant, "A comprehensive review of background subtraction algorithms evaluated with synthetic and real videos," *Computer Vision and Image Understanding*, vol. 122, pp. 4–21, 2014.

[13] B. Garcia-Garcia, T. Bouwmans, and A. J. R. Silva, "Background subtraction in real applications: Challenges, current models and future directions," *Computer Science Review*, vol. 35, p. 100204, 2020.

[14] Z. Zivkovic and F. Van Der Heijden, "Efficient adaptive density estimation per image pixel for the task of background subtraction," *Pattern recognition letters*, vol. 27, no. 7, pp. 773–780, 2006.

[15] A. Ghodrati, B. E. Bejnordi, and A. Habibian, "Frameexit: Conditional early exiting for efficient video recognition," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 15608–15618.

[16] A. Jadon, M. Omama, A. Varshney, M. S. Ansari, and R. Sharma, "FireNet: A specialized lightweight fire & smoke detection model for real-time IoT applications," *arXiv preprint arXiv:1905.11922*, 2019.

[17] "Firesense." Available: https://www.firesense.eu/

[18] M. T. Cazzolato *et al.*, "Fismo: A compilation of datasets from emergency situations for fire and smoke analysis," in *Brazilian symposium on databases-SBBD*, SBC Uberlândia, Brazil, 2017, pp. 213–223.

[19] C. R. Steffens, R. N. Rodrigues, and S. S. da Costa Botelho, "An unconstrained dataset for non-stationary video based fire detection," in *2015 12th latin american robotics symposium and 2015 3rd brazilian symposium on robotics (LARS-SBR)*, IEEE, 2015, pp. 25–30.

[20] P. V. A. de Venâncio, A. C. Lisboa, and A. V. Barbosa, "An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices," *Neural Computing and Applications*, vol. 34, no. 18, pp. 15349–15368, 2022.

[21] E. Cetin, "Computer vision based fire detection software." 2015. Available: http://signal.ee.bilkent.edu.tr/VisiFire/

[22] B. C. Ko, S. J. Ham, and J. Y. Nam, "Modeling and formalization of fuzzy finite automata for detection of irregular fire flames," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 12, pp. 1903–1912, 2011.

[23] P. Foggia, A. Saggese, and M. Vento, "Real-time fire detection for video-surveillance applications using a combination of experts based on color, shape, and motion," *IEEE TRANSACTIONS on circuits and systems for video technology*, vol. 25, no. 9, pp. 1545–1556, 2015.

[24] G. Lin, Y. Zhang, Q. Zhang, Y. Jia, G. Xu, and J. Wang, "Smoke detection in video sequences based on dynamic texture using volume local binary patterns." *KSII Trans. Internet Inf. Syst.*, vol. 11, no. 11, pp. 5522–5536, 2017.

[25] "AI-hub." Available: https://aihub.or.kr/aihubdata/data/view.do?dataSetSn=71751

[26] L. Huang, G. Liu, Y. Wang, H. Yuan, and T. Chen, "Fire detection in video surveillances using convolutional neural networks and wavelet transform," *Engineering Applications of Artificial Intelligence*, vol. 110, p. 104737, 2022.

[27] O. Önal and E. Dandıl, "Video dataset for the detection of safe and unsafe behaviours in workplaces," *Data in Brief*, vol. 56, p. 110791, 2024.

[28] D. Deniz, E. Ros, E. M. Ortigosa, and F. Barranco, "Optimized edge-cloud system for activity monitoring using knowledge distillation," *Electronics*, vol. 13, no. 23, p. 4786, 2024.

[29] M. Rohrbach *et al.*, "Recognizing fine-grained and composite activities using hand-centric features and script data," *International Journal of Computer Vision*, vol. 119, no. 3, pp. 346–373, 2016.

[30] R. Marroquin, J. Dubois, and C. Nicolle, "WiseNET: An indoor multi-camera multi-space dataset with contextual information and annotations for people detection and tracking," *Data in brief*, vol. 27, p. 104654, 2019.

[31] C. R. Steffens, R. N. Rodrigues, and S. S. da Costa Botelho, "Non-stationary VFD evaluation kit: Dataset and metrics to fuel video-based fire detection development," in *Latin american robotics symposium*, Springer, 2016, pp. 135–151.

[32] D. Mukhopadhyay, R. Iyer, S. Kadam, and R. Koli, "FPGA deployable fire detection model for real-time video surveillance systems using convolutional neural networks," in *2019 global conference for advancement in technology (GCAT)*, IEEE, 2019, pp. 1–7.

[33] G. Jocher, "YOLOv5 by Ultralytics." May 2020. doi: 10.5281/zenodo.3908559. Available: https://github.com/ultralytics/yolov5

[34] "Timm/hgnetv2_b5.ssld_stage2_ft_in1k · hugging face." Sep. 2025. Available: https://huggingface.co/timm/hgnetv2_b5.ssld_stage2_ft_in1k

[35] R. Wightman, "PyTorch image models (timm)." 2019. Available: https://github.com/rwightman/pytorch-image-models

[36] C. Cui *et al.*, "Beyond self-supervision: A simple yet effective network distillation alternative to improve backbones," *arXiv preprint arXiv:2103.05959*, 2021.

[37] "PaddleClas/docs/en/models/PP-HGNetV2_en.md at f1233c18455b8acde4fc42ab0bea575fa06daa8e · PaddlePaddle/PaddleClas." Available: https://github.com/PaddlePaddle/PaddleClas/blob/f1233c18455b8acde4fc42ab0bea575fa06daa8e/docs/en/models/PP-HGNetV2_en.md?plain=1#L33

[38] J. Hestness *et al.*, "Deep learning scaling is predictable, empirically," *arXiv preprint arXiv:1712.00409*, 2017.

[39] I. M. Alabdulmohsin, B. Neyshabur, and X. Zhai, "Revisiting neural scaling laws in language and vision," *Advances in Neural Information Processing Systems*, vol. 35, pp. 22300–22312, 2022.

[40] Y. Bahri, E. Dyer, J. Kaplan, J. Lee, and U. Sharma, "Explaining neural scaling laws," *Proceedings of the National Academy of Sciences*, vol. 121, no. 27, p. e2311878121, 2024.